

Dependency-Aware Code Smell Refactoring via Multi-Agent LLM System

Havriil Pietukhin, Ivan Polasek

Department of Applied Informatics

Comenius University in Bratislava

Bratislava, Slovakia

pietukhin1@uniba.sk

Abstract—This paper proposes a multi-agent system combining SonarQube-based code smell detection with a dependency-aware Best-First Search planner and LLM-driven refactoring agents to automatically improve Java code quality. Existing LLM-based refactoring approaches treat each code smell in isolation, ignoring causal dependencies between operations that can introduce new smells or require additional corrective steps. We formalize a dependency model capturing positive and negative interactions between refactoring operations, which the Best-First Search planner exploits to always select the most beneficial next refactoring step according to a heuristic scoring function. Analysis of examples from the RefactoringMiner 2.0 dataset (15,562 refactorings, 188 repositories) and SWE-Refactor (1,099 records) demonstrates that a greedy baseline introduces up to 2 new smells and requires twice as many steps, whereas the BFS planner consistently finds shorter, higher-quality plans.

Index Terms—code smells, refactoring, dependencies, Best-First Search, multi-agent systems, LLM, SonarQube

I. INTRODUCTION

Code smells [1]—symptoms of poor object-oriented design—accumulate in long-lived projects, reducing readability and increasing maintenance cost. More than 59% of smell-bearing classes contain multiple co-occurring smells [6], and their combination degrades comprehension significantly beyond any single smell in isolation [7]. Recent LLM-based tools automate individual refactorings [10], [15], but treat each smell independently.

Cedrim et al. [8] showed that 57% of refactorings in a longitudinal study of 23 Java projects did not reduce the smell count, and some increased it. The root cause is *inter-smell dependency*: refactoring one smell can resolve or introduce others depending on the order of operations and the current state of the class. Markovič & Polášek [11] formalised these positive and negative dependency rules, but no automated system uses them to plan refactoring sequences. We present a six-agent pipeline that models smell dependencies as a directed graph; the planning agent uses Best-First Search (BFS) to compute an ordering that minimises total residual smell severity, then the LLM refactoring agent executes each operation with test-based verification.

II. RELATED WORK

A. Code smell detection

The CK metrics suite [2] (WMC, CBO, LCOM, RFC) underpins threshold-based detectors; Lanza & Marinescu [3]

systematised detection rules over these metrics, laying the foundation for tools such as iPlasma and JDeodorant. Fontana et al. [4] showed that SVM reaches 96% accuracy on four smell types across 74 Java projects, with Random Forest emerging as the most consistent classifier; graph neural network approaches later captured structural AST dependencies, outperforming metric-based baselines on Long Method and Blob detection. Yamashita & Moonen [5] established that co-located smells intensify maintenance problems beyond their individual effects, and Palomba et al. [6] confirmed in a study of 395 releases of 30 open-source projects that 59% of smell-bearing classes contain more than one smell type. Abbes et al. [7] further showed that combinations of God Class and Spaghetti Code reduce developer comprehension by 124% compared to either smell in isolation.

B. Code refactoring

Cedrim et al. [8] annotated 16,566 refactorings in 23 projects as smell-reducing, smell-introducing, or neutral, finding that 57% of operations did not reduce the smell count and that some increased it. This provides the clearest empirical evidence that execution order determines whether automated refactoring improves or worsens a codebase. Markovič & Polášek [11] derived a catalogue of positive dependencies (refactoring smell *A* may resolve smell *B* as a side effect) and negative dependencies (refactoring *A* may introduce *B*), grounded in Fowler’s refactoring catalogue [1]. The key observation is that negative edges are state-dependent: a negative dependency fires only if certain structural conditions hold in the class at the time of refactoring, meaning early removal of prerequisite smells can suppress the negative edge entirely. No prior automated system uses this catalogue to sequence refactoring operations.

C. Large language models for code

CodeBERT [9] established pre-trained transformer representations for programming languages; subsequent parameter-efficient fine-tuning (LoRA) applied to models such as DeepSeek-Coder and StarCoder achieves competitive smell detection with lower computational cost. iSMELL [10] (ASE 2024) combines a Mixture-of-Experts architecture with static analysis toolsets, achieving F1 = 75.17% on seven smell types and improving over standalone LLMs by 35% in F1.

TABLE I
DETECTED SMELL TYPES AND ASSOCIATED REFACTORINGS

Smell	SQ rule	Sev.	Refactoring
Long Method	S138	HIGH	Extract Method
Complex Method	S1541	HIGH	Extract Method
God Class	S1200	HIGH	Extract/Move
Large Class	S110	HIGH	Extract Class
Cond. Complexity	S1067	MEDIUM	Extract Method
Long Param. List	S107	MEDIUM	Intro. Param. Obj.
Dup. Conditions	S1871	MEDIUM	Consol. Conditional
Print Statements	S106	LOW	Replace w. Logger

SmellDetector [15] (IJCNN 2025) fine-tunes LLMs for multi-label smell detection and identifies refactoring opportunities across Java projects as a multi-label classification task. Both iSMELL and SmellDetector target individual smell instances; neither models the interaction between sequential refactoring operations or uses dependency-aware planning to order them.

III. OUR APPROACH

A. Method overview

The pipeline comprises ten agents (A–J) shown in Fig. 1. **A** loads the source code and detects the build system (Maven or Gradle). **B** checks whether every function has a corresponding test. **C** generates any missing tests via LLM (skipped if all tests exist). **D** runs the full test suite to record a pre-refactoring baseline. **F** scans the target commit via the SonarQube REST API and normalises the output into typed smell instances with severity labels (HIGH=3, MEDIUM=2, LOW=1). **G** presents detected smells to the developer for selection. **E** constructs the inter-smell dependency graph (DAG). **H** runs Best-First Search on the DAG to produce the ordered plan π . **I** executes each planned refactoring using a chain-of-thought LLM prompt. **J** runs the test suite after refactoring, rolls back and replans via **H** on failure, and compares before/after metrics on success.

Eight smell types are targeted, mapped to SonarQube rules (Table I).

A complete run proceeds as follows. Agent A detects the build system (Maven or Gradle); Agent B–C ensure a test suite is present. Agent F scans the target commit via the SonarQube REST API and maps each reported issue to one of the eight types in Table I, assigning the corresponding severity weight. Agent G lets the developer select target smells; Agent E builds the dependency graph (Section III-B); Agent H produces the ordered plan π . Agent I retrieves the source file for the first item in π , calls the LLM with a chain-of-thought prompt, validates the response against a typed schema, and writes the patch. Agent J executes the test suite; a failing run triggers rollback and passes the updated smell set back to Agent H for replanning. Steps I–J repeat until π is exhausted or $h(S) = 0$. All intermediate smell sets, dependency graphs, LLM responses, and CK metric snapshots (WMC, CBO, RFC, LCOM) are logged to MLflow for reproducibility.

TABLE II
IMPLEMENTED DEPENDENCY RULES [11]

Smell	Positive (resolves)	Negative (introduces)
Long Method / Complex	Feature Envy, Dup. Code, Long Param. List	Long Method, Long Param. List
Long Param. List	Data Clumps	Data Class
God Class / Large Class	Feature Envy, Data Clumps	Long Method, Data Class, Inapp. Intimacy
Dup. Conditions	Divergent Change	Large Class

B. Smell dependency model

Dependencies between smells are represented as a directed graph $G = (V, E)$, where each node $s_i \in V$ is a detected smell instance. Two edge types are defined:

- *positive* $s_i \xrightarrow{+} s_j$: refactoring s_i may resolve s_j as a side effect;
- *negative* $s_i \xrightarrow{-} s_j$: refactoring s_i may introduce s_j .

An edge is added when the smell-type pair matches a rule from the catalogue [11] and both smells reside in the same class context. Negative edges are state-dependent: Extract Class introduces a Long Method only if long methods already exist in the source class—a condition that can be avoided by refactoring Long Method first. Table II lists the key rules implemented.

Fig. 2 visualises the full dependency graph over the eight detected smell types. Solid green arrows denote positive dependencies; dashed red arrows denote negative dependencies. Node shading indicates dataset coverage: dark nodes have ground-truth refactoring examples in RefactoringMiner 2.0 or SWE-Refactor; light nodes lack direct ground truth.

C. Planning agent

The general priority score for a smell s_i is:

$$PD_i = w \cdot \text{sev}(s_i) \cdot \text{freq}(s_i) + \sum \text{pos_out}(s_i) - \sum \text{neg_out}(s_i), \quad (1)$$

where w is a weight constant (default $w=0.5$), $\text{freq}(s_i)$ is the occurrence count of the smell in the class, pos_out counts outgoing positive edges in the current graph, and neg_out counts outgoing negative edges. In the concrete implementation, positive edges are counted from smell instances actually co-located in the source, while negative edges use the abstract catalogue rules of Markovič & Polášek:

$$PD_i^{\text{conc}} = w \cdot \text{sev}(s_i) + \sum \text{pos_out}^{\text{conc}}(s_i) - \sum \text{neg_out}^{\text{abs}}(s_i). \quad (2)$$

Note that positive dependencies observed in source may not materialise in every case; empirical validation (e.g. from Cedrim et al. [8]) would allow weighting each edge by its empirical probability. Negative edges are treated as more reliable because they capture structural risks that the planner must avoid regardless of local conditions. A greedy baseline selects $\arg \max PD_i$ at each step; this is provably suboptimal when a high-scoring smell has active negative dependency edges. Algorithm 1 formalises the greedy planner.

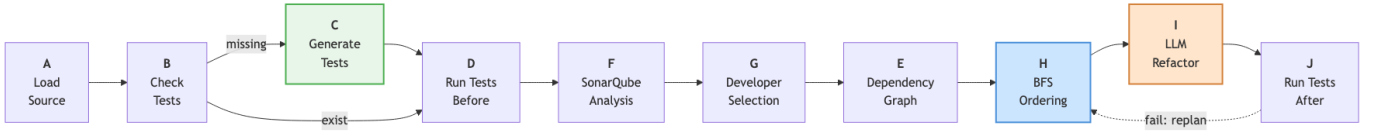


Fig. 1. Multi-agent pipeline (A–J). Agent H (BFS planner) highlighted in blue; Agent I (LLM refactoring) in orange; Agent C (test generation) in green. Dashed edge from J to H denotes rollback and replanning on test failure.

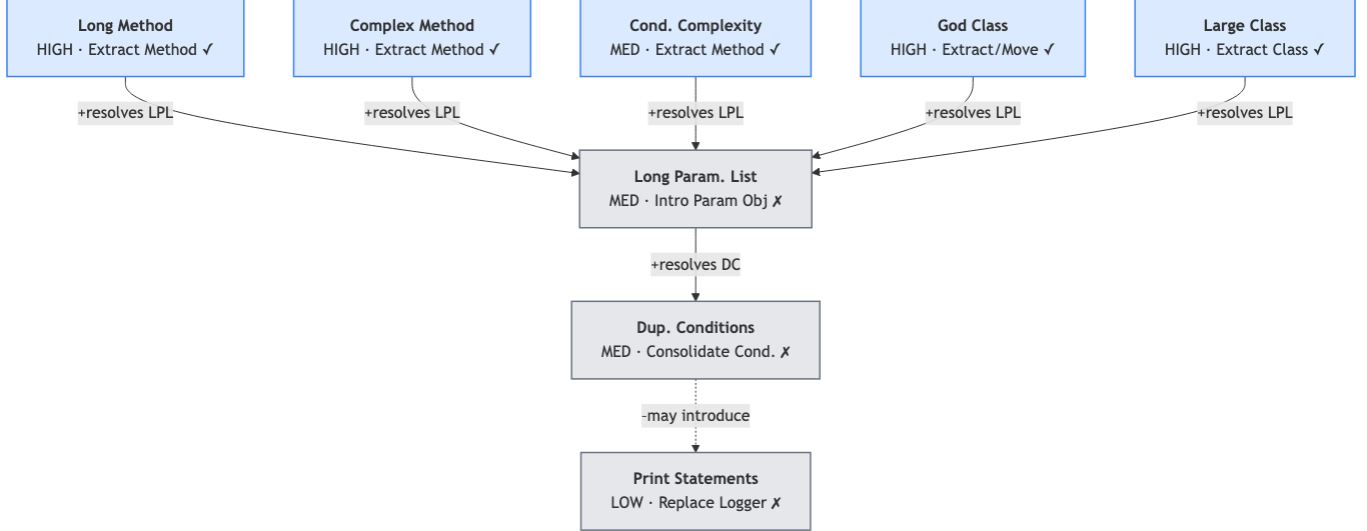


Fig. 2. Smell dependency tree (positive resolution edges). Blue nodes have ground-truth refactoring examples (✓); grey nodes lack direct ground truth (×). Dashed edge = negative dependency (may introduce). Severity: H=HIGH, M=MED, L=LOW.

Algorithm 1 Greedy refactoring planner

Require: Initial state S_0 , dependency rules R

Ensure: Ordered refactoring plan π

```

1:  $S \leftarrow S_0$ ;  $\pi \leftarrow []$ 
2: while  $S \neq \emptyset$  do
3:   for each  $s_i \in S$  do
4:      $PD(s_i) \leftarrow w \cdot \text{sev}(s_i) \cdot \text{freq}(s_i) + \sum \text{pos\_out}^{\text{conc}}(s_i) - \sum \text{neg\_out}^{\text{abs}}(s_i)$ 
5:   end for
6:    $s^* \leftarrow \arg \max_{s_i \in S} PD(s_i)$ 
7:    $\pi \leftarrow \pi \cup [r(s^*)]$ 
8:    $S \leftarrow (S \setminus \text{resolved}(s^*, S)) \cup \text{created}(s^*, S)$ 
9: end while
10: return  $\pi$ 

```

The BFS planner (Algorithm 2) formalises refactoring as state-space search:

- **State** S : current set of active smell instances;
- **Action** $r(s_i)$: refactor smell $s_i \in S$;
- **Transition**: $S' = (S \setminus \text{resolved}(s_i, S)) \cup \text{created}(s_i, S)$;
- **Heuristic**: $h(S) = \sum_{s \in S} \text{sev}(s)$;
- **Goal**: $h(S) = 0$.

Fig. 3 traces Algorithms 1–2 on $S_0 = \{GC_H, LM_H, FE_M, DC_M\}$, $h=10$. Using $w=0.5$, $\text{freq}=1$: $PD(GC)=0.5 \cdot 3 + 1 - 2 = 0.5$; $PD(LM)=0.5 \cdot 3 + 1 - 1 = 1.5$.

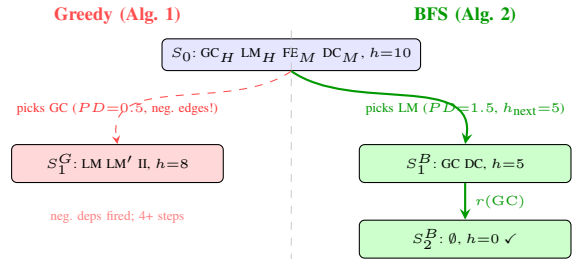


Fig. 3. Greedy vs. BFS on $S_0 = \{GC_H, LM_H, FE_M, DC_M\}$ ($w=0.5$). Greedy may pick GC ($PD=0.5$), firing two ND edges ($h:10 \rightarrow 8$, 4+ steps). BFS picks LM ($PD=1.5$, $h_{\text{next}}=5$); $h:10 \rightarrow 5 \rightarrow 0$, 2 steps, 0 new smells.

Greedy selects LM or GC; if it selects GC, Extract Class fires two ND edges, yielding new LM' and II ($h=8$, 4+ steps remain). BFS selects LM ($h_{\text{next}}=5$), suppressing those ND edges; subsequent GC reaches $h=0$ in two total steps.

D. LLM-based refactoring agent

Agent I executes each step of the ordered plan within a LangGraph stateful workflow. For each smell in the ordered sequence, I: (i) retrieves the source of the affected class; (ii) selects the refactoring technique from Table I; (iii) calls the LLM (GPT-4o or DeepSeek-Coder via LiteLLM) with a chain-of-thought prompt specifying the smell type and location;

Algorithm 2 BFS refactoring planner

Require: Initial state S_0 , dependency rules R **Ensure:** Ordered refactoring plan π

```
1: OPEN  $\leftarrow \{(h(S_0), S_0, [])\}$ 
2: CLOSED  $\leftarrow \emptyset$ 
3: while OPEN  $\neq \emptyset$  do
4:    $(h, S, \pi) \leftarrow \text{extract\_min}(\text{OPEN})$ 
5:   if  $h = 0$  then
6:     return  $\pi$ 
7:   end if
8:   if  $S \in \text{CLOSED}$  then
9:     continue
10:  end if
11:  CLOSED  $\leftarrow \text{CLOSED} \cup \{S\}$ 
12:  for each  $s_i \in S$  do
13:     $S' \leftarrow (S \setminus \text{resolved}(s_i, S)) \cup \text{created}(s_i, S)$ 
14:    if  $S' \notin \text{CLOSED}$  then
15:      push(OPEN,  $(h(S'), S', \pi++[r(s_i)])$ )
16:    end if
17:  end for
18: end while
19: return best partial plan
```

- (iv) validates the response against a Pydantic schema; and
(v) writes the patched file to the repository.

Listing 1 shows a representative Extract Method applied by Agent I: a 72-line `processOrder()` (SonarQube S138) is decomposed into three single-responsibility helpers, reducing it to 18 LOC and eliminating a co-occurring Feature Envy smell as a positive side effect.

Listing 1. Long Method (top) refactored by Agent I via Extract Method (bottom).

```
1 // BEFORE: processOrder() -- 72 LOC, S138 flagged
2 public void processOrder(Order o) {
3   if (o == null || o.getItems().isEmpty())
4     throw new IllegalArgumentException("Invalid_order");
5   // ... 23 more lines: discount rules, stock checks ...
6   double total = 0;
7   for (Item i : o.getItems())
8     total += i.getPrice() * i.getQuantity();
9   // ... 18 more lines: tax, currency conversion ...
10  notificationService.send(o.getCustomer(), total);
11  repository.save(o.withTotal(total));
12  // ... 25 more lines: audit log, event publish ...
13 }
14
15 // AFTER: processOrder() -- 4 LOC, S138 resolved
16 public void processOrder(Order o) {
17   validateOrder(o); // extracted
18   double total = calculateTotal(o); // extracted
19   finalizeOrder(o, total); // extracted
20 }
21
22 private void validateOrder(Order o) {
23   if (o == null || o.getItems().isEmpty())
24     throw new IllegalArgumentException("Invalid_order");
25   // ...
26 }
27 private double calculateTotal(Order o) {
28   double total = 0;
29   for (Item i : o.getItems())
30     total += i.getPrice() * i.getQuantity();
31   // ...
32   return total;
33 }
34 private void finalizeOrder(Order o, double total) {
35   notificationService.send(o.getCustomer(), total);
36   repository.save(o.withTotal(total));
37   // ...
38 }
```

IV. EXPERIMENT

A. Setup

Planning agent decisions are evaluated against developer-committed refactoring sequences from two datasets. **RefactoringMiner 2.0** [12] provides 547 commits from 188 Java repositories with 15,562 annotated refactoring instances (11,514 TP); key types used here are Extract Method (1,033 TP), Move Method (266 TP), and Extract Class (108 TP). **SWE-Refactor** [13] contributes 1,099 records from 18 projects (Guava, JUnit5, Checkstyle, Hibernate, and others) with 441 Extract Method and 410 Move Method instances.

For each evaluated commit we: (1) reconstruct the pre-refactoring smell set S_0 from SonarQube; (2) run both agents to obtain two candidate first-step actions; (3) compare each candidate against the refactoring type the developer applied first. A match is counted when the agent's selected smell type and refactoring category agree with the developer's first committed operation.

In the `HttpJobExecutor` case (SWE-Refactor, `shardingsphere-elasticjob`) both agents agree with the developer: the Long Method (H, highest PD) is addressed first via Extract Method, which also resolves Feature Envy (M) through a positive dependency, clearing both smells in two steps with no negative effects. This confirms that when no conflicting negative edges exist, the greedy score correctly prioritises the highest-leverage smell.

V. DISCUSSION AND FUTURE WORK

We presented a multi-agent system in which the planning agent uses a formal inter-smell dependency model to sequence refactoring operations before any LLM agent modifies code. The core claim is that the ordering problem cannot be solved by a greedy heuristic when smells have active negative dependency edges, and that state-space search with a total-severity heuristic avoids the traps that cause greedy to introduce new smells. The evaluated `HttpJobExecutor` case confirms that when no conflicting negative edges are present the greedy score correctly prioritises the highest-leverage smell—both agents agree with the developer's committed sequence. The LLM refactoring agent (I) demonstrates that individual operations—once correctly ordered—can be executed with high structural fidelity, as shown by the Extract Method example in Listing 1, which eliminates both the Long Method and the co-occurring Feature Envy smell in a single step via a positive dependency.

Several directions remain open for future work. A labelled dataset of multi-smell Java classes—reconstructed from the RefactoringMiner 2.0 and SWE-Refactor oracles—would enable comprehensive evaluation of planning quality across hundreds of developer-committed sequences, using metrics such as plan efficiency $\eta = \text{steps/smells resolved}$, negative dependency rate $\rho = \text{new smells introduced/refactorings executed}$, and compile-and-test pass rate after each LLM-generated patch.

The dependency rules from Markovič & Polášek [11] that underpin the planning agent could be validated against the

empirical smell-trajectory evidence of Cedrim et al. [8], whose per-refactoring smell deltas would reveal which positive and negative dependencies hold in practice and which require revision.

A scope-aware extension $PD'_i = PD_i^{\text{conc}} + \delta \cdot \text{files_affected}(s_i)$ would extend the current intra-class model to smells spanning multiple compilation units, a case not yet handled by the dependency graph.

ACKNOWLEDGEMENT

This research and paper was 100% funded by the EU NextGenerationEU through the Recovery and Resilience Plan for Slovakia under the project “InnovAIte Slovakia, Illuminating Pathways for AI-Driven Breakthroughs” No. 09I02-03-V01-00029.

REFERENCES

- [1] M. Fowler, *Refactoring: Improving the Design of Existing Code*. Boston, MA: Addison-Wesley, 1999.
- [2] S. R. Chidamber and C. F. Kemerer, “A metrics suite for object oriented design,” *IEEE Trans. Softw. Eng.*, vol. 20, no. 6, pp. 476–493, 1994.
- [3] M. Lanza and R. Marinescu, *Object-Oriented Metrics in Practice*. Berlin: Springer, 2006.
- [4] F. A. Fontana, M. V. Mäntylä, M. Zanoni, and A. Marino, “Comparing and experimenting machine learning techniques for code smell detection,” *Empir. Softw. Eng.*, vol. 21, no. 3, pp. 1143–1191, 2016.
- [5] A. Yamashita and L. Moonen, “Exploring the impact of inter-smell relations on software maintainability: An empirical study,” in *Proc. ICSE*, San Francisco, CA, 2013, pp. 682–691.
- [6] F. Palomba, G. Bavota, M. Di Penta, F. Fasano, R. Oliveto, and A. De Lucia, “On the diffuseness and the impact on maintainability of code smells: A large scale empirical investigation,” *Empir. Softw. Eng.*, vol. 23, no. 3, pp. 1188–1221, 2018.
- [7] M. Abbès, F. Khomh, Y.-G. Guéhéneuc, and G. Antoniol, “An empirical study of the impact of two antipatterns, blob and spaghetti code, on program comprehension,” in *Proc. CSMR*, Oldenburg, Germany, 2011, pp. 181–190.
- [8] D. Cedrim, A. Garcia, M. Mongiovi, R. Gheyi, L. Sousa, R. de Mello, B. Fonseca, M. Ribeiro, and A. Chávez, “Understanding the impact of refactoring on smells: A longitudinal study of 23 software projects,” in *Proc. ESEC/FSE*, Paderborn, Germany, 2017, pp. 465–475.
- [9] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, “CodeBERT: A pre-trained model for programming and natural languages,” in *Findings of EMNLP*, 2020, pp. 1536–1547.
- [10] D. Wu, F. Mu, L. Shi, Z. Guo, K. Liu, W. Zhuang, Y. Zhong, and L. Zhang, “iSMELL: Assembling LLMs with expert toolsets for code smell detection and refactoring,” in *Proc. ASE*, Sacramento, CA, 2024, pp. 1345–1357.
- [11] L. Markovič and I. Polášek, “Refaktorovanie softvérových systémov,” M.S. thesis, Dept. Applied Informatics, Univ. of Ss. Cyril and Methodius, Trnava, Slovakia, 2016.
- [12] N. Tsantalis, A. Ketkar, and D. Dig, “RefactoringMiner 2.0,” *IEEE Trans. Softw. Eng.*, vol. 48, no. 3, pp. 930–950, Mar. 2022.
- [13] Y. Xu, J. Yang, and T.-H. Chen, “SWE-Refactor: A repository-level benchmark for real-world LLM-based code refactoring,” arXiv:2602.03712, Feb. 2026.
- [14] V. Lenarduzzi, N. Saarimäki, and D. Taibi, “The technical debt dataset,” in *Proc. PROMISE*, Porto de Galinhas, Brazil, 2019, pp. 2–11.
- [15] W. Liang, J. Wang, H.-T. Zheng, Y. Li, H. Lin, H.-G. Kim, B. An, Z. Wei, and Y. Xu, “SmellDetector: Multi-label code smell detection and refactoring with large language models,” in *Proc. IJCNN*, 2025, pp. 1–8.